Algorithms and Computability - Project Report

Remigiusz Iwanik Bartosz Kaczmarek Jakub Muszyński
Ignacy Walużenicz

October 2025

1 Introduction

Graphs – in particular *directed multigraphs* with parallel arcs and self-loops – are a natural model for transportation, communication, biochemical, and workflow systems. In these settings, we are often confronted with three tightly related questions: (i) how to compare two networks; (ii) whether one network embeds into another; and (iii) if not, how to *minimally* augment the host so that the embedding becomes possible. This report studies these questions under a unified framework tailored to directed multigraphs.

Problem setting. Given two (multi)graphs $G = (V, E)$ and $H = (U, F)$ with $|V| \leq |U|$, we address:

1. A size measure that reflects both structural scope and connectivity. We adopt $|G| = |V(G)| + |E(G)|$ with $|E(G)| = \sum_{u,v} \mathrm{mult}_{\mathrm{G}}\big((u, v)\big)$, i.e., edges counted with multiplicity.

2. A principled distance between graphs. We use *Graph Edit Distance* (GED) (Sanfeliu and Fu 1983) as the primary notion of dissimilarity and relate it to an injective, multiplicity-mismatch objective that directly underpins our minimal-extension formulation.

3. Subgraph detection and enumeration. We ask whether there exists an injective mapping $\phi : V(G) \to V(H)$ such that for all $u, v \in V(G)$ it is true that $\mathrm{mult}_{\mathrm{G}}\big((u, v)\big) \leq \mathrm{mult}_{\mathrm{H}}\Big(\big(\phi(u), \phi(v)\big)\Big)$, and we consider the task of enumerating *up to* N distinct such embeddings.

4. Minimal extension. When G is not a subgraph of H, we determine the smallest augmentation $H' \supseteq H$ that admits an embedding of G. We further study the variant that requires *simultaneously* supporting at least N embeddings (the N-*minimal extension* problem).

Approach and contributions.

- **Foundations.** We formalize size and distance for directed multigraphs, using GED as the primary distance and introducing a *deficit* view

$$\text{deficit}(\phi) = \sum_{u,v \in V(G)} \max\Big(0,\, \text{mult}_{\text{G}}\left((u,v)\right) - \text{mult}_{\text{H}}\left(\big(\phi(u),\phi(v)\big)\right)\Big),$$

  which quantifies the edge additions needed by a particular injective mapping. This directly yields the minimal extension H' and extends to the N-minimal case via elementwise max aggregation of mapping-induced deficits.

- **Exact algorithms.** Building on Ullmann's backtracking method for subgraph isomorphism (Ullmann 1976), we adapt feasibility checks to multiplicities for (a) deciding subgraph existence and enumerating embeddings *up to* N, and (b) computing deficit patterns per mapping. For N-minimal extension, we combine these deficits to find the smallest joint augmentation; optional branch-and-bound pruning reduces the search where applicable.

- **Approximation strategies.** To cope with large instances, we develop an approximation approach: a *selection-function* approach that evaluates only K promising injective mappings. For constructing approximate extensions, we union the deficits over the selected K mappings. As a deterministic selection mechanism, we instantiate a degree-/multiplicity-aware *K-best assignments* scheme using the Hungarian algorithm (Kuhn 1955) and Murty's method (Murty 1968).

- **Complexity view.** Subgraph isomorphism is NP-complete (listed as GT48 in Garey and Johnson 1979), counting embeddings is #P-complete (Valiant 1979), and the minimal-extension optimization is NP-hard. Our exact procedures therefore target small to moderate graphs, while the approximation pipelines offer practical performance for larger inputs with controllable accuracy via K.

Scope and assumptions. Unless stated otherwise, we work with directed multigraphs (parallel arcs and self-loops permitted). Labels and operation-specific edit costs can be incorporated as constraints or penalties in GED, but are not required by the core methods developed here.

2 Mathematical Foundations

2.1 Graphs

A **graph** G is a structure consisting of two sets: $V(G)$, which is a finite set of vertices, and $E(G) \subseteq V(G) \times V(G)$, which is a finite set of edges of the graph G. The **edge** $e \in E(G)$ is an unordered pair of vertices: $e = \{v, u\}$ for $v, u \in V(G)$. The degree function $\deg_{\mathrm{G}}(v)$ is defined as the number of edges incident to vertex $v \in V(G)$.

A **directed graph** G is an extension to the concept of a graph, in which edges also have orientations. Typically, a **directed edge**, also known as an **arc**, is represented by an ordered pair: $e = (u, v)$ for $u, v \in V(G)$. In this case, u is called the **tail** of the edge e, and v is the **head** of the edge e. In case of directed graphs, aside from the degree function $\deg_{\mathrm{G}}(v)$, we also define the indegree function $\mathrm{indeg}_{\mathrm{G}}(v)$ and outdegree funcion $\mathrm{outdeg}_{\mathrm{G}}(v)$, which are the number of incident edges for which the given vertex is a head or a tail respectively.

A **multigraph** G is another extension to the concept of a graph, in which any two vertices can be joined by multiple edges. In this case, $E(V)$ is typically represented either as a multiset of edges (or arcs in case of a directed multigraph). This definition of a graph assumes edges have no identity - that is, edges which join the same vertices (and with the same orientation in case of directed multigraphs) are indistinguishable from one another. Multiplicity $\mathrm{mult}_{\mathrm{G}}(e)$ of an edge $e \in E(G)$ is the number of singular edges joining the same (ordered) pair of vertices. Thus, an alternative representation of a multigraph's set of edges would be a set of ordered pairs $\big(e, \mathrm{mult}_{\mathrm{G}}(e)\big)$. Note however, that in this representation, we shall interpret the size of the set of edges $|E(G)|$ as $\sum_{e \in E(G)} \mathrm{mult}_{\mathrm{G}}(e)$ Degree, indegree and outdegree for multigraphs are redefined as the sums of multiplicities of the counted edges.

In this report, we will deal with the most general concept of the mentioned ones, which is a directed multigraph. Thus, for brevity, henceforth the use of words graph G and set of edges $E(G)$ are to be understood as an directed multigraph and multiset of arcs respectively.
To summarise the notation used:

1. G, H, H' - a directed multigraph.
2. $V(G)$ - set of vertices of G.
3. $E(G)$ - a multiset of edges of G.
4. $\deg_{\mathrm{G}}(v)$, $\mathrm{indeg}_{\mathrm{G}}(v)$, $\mathrm{outdeg}_{\mathrm{G}}(v)$ for $v \in V(G)$ - the degree, indegree and outdegree of vertex v.
5. $\mathrm{mult}_{\mathrm{G}}(e)$ for $e = (u, v)$, $u, v \in V(G)$ - the multiplicity of the edge e.

2.2 Size of a graph

We define the size of the graph $|G|$ as the sum of the number of vertices $\big|V(G)\big|$ and the number of edges $\big|E(G)\big|$: $|G| = \big|V(G)\big| + \big|E(G)\big|$. The definition is

compatible with the notion of a measure:

1. The size of an empty graph $\varnothing$ is 0.
2. As the number of edges and vertices in a graph is always nonnegative, the size of a graph is also nonnegative.
3. The size of a union of fully disjoint graphs is equal to the sum of their sizes.

2.2.1 Pseudocode

```
1: function GraphSize(G)
2:     v ← |V(G)|
3:     e ← 0
4:     for all ordered pairs {u, v} ⊆ V(G) such that u ≠ v do
5:         e ← e + mult_G((u, v))
6:     end for
7:     for all vertices u ∈ V(G) do
8:         e ← e + mult_G((u, u))
9:     end for
10:    return v + e
11: end function
```

2.3 Distance between graphs

We define the distance between two graphs G and H as a **graph edit distance** $\operatorname{ged}(G, H)$, which is the length of the minimal edit path which transforms G into G' such that $G' \simeq H$. An **edit path** is an ordered sequence of graph edit operations. In our case, we define the graph edit operations as:

1. vertex addition ($\operatorname{addv}(v)$),
2. vertex deletion ($\operatorname{delv}(v)$),
3. edge addition ($\operatorname{adde}(e)$),
4. edge deletion ($\operatorname{dele}(e)$).

(Note that it is not the only possible set of operations, and we merely adopt the minimal set required for it to be useful to us.) We propose that such distance is a proper metric. For that to be true, $\operatorname{ged}(G, H)$ must satisfy the following properties:

1. $\operatorname{ged}(G, H) = 0 \iff G \simeq H$
2. $\operatorname{ged}(G, H) > 0 \iff G \not\simeq H$
3. $\operatorname{ged}(G, H) = \operatorname{ged}(H, G)$
4. $\forall_{F,G,H} \operatorname{ged}(F, G) + \operatorname{ged}(G, H) \geq \operatorname{ged}(F, H)$

2.3.1 $\text{ged}\,(G, H) = 0 \iff G \simeq H$

As the G and H are already isomorphic, the minimum edit path is empty and hence its length is 0.

2.3.2 $\text{ged}\,(G, H) > 0 \iff G \not\simeq H$

Conversely, if G and H are not isomorphic, the minimum edit path must include at least one operation to construct graph G' isomorphic to H', and hence its length is greater than 0.

2.3.3 $\text{ged}\,(G, H) = \text{ged}\,(H, G)$

We note that for every of the four provided graph edit operations, there exists an operation with opposite effect. Thus, suppose that there exists minimal edit path p_G such that after applying its operations on G, we find G' isomorphic to H. Then, we can always construct an edit path transforming H into H' isomorphic to G by the following procedure:

1. Begin with an empty empty edit path $p_H = ()$
2. Iterate overp_G in reverse order, with the current operation being $o(x)$.
   (a) For the current operation $o(x)$, find the opposite operation:
      - $\text{addv}(x) \iff \text{delv}(x)$
      - $\text{adde}(x) \iff \text{dele}(x)$
   (b) Append the opposite operation to the end of p_H

As we can see, the length of p_H will always be equal to the length of p_G and we know that it constructs H' isomorphic to G. Now, suppose there were a different path p'_H which achieves the same, but is shorter. That would mean that we can construct an edit path p'_G, by using the above procedure on p'_H, which is of the same shorter length and constructs G' isomorphic to H. However, p_G is already minimal by our assumption, hence this would be a contradiction. Therefore, p_H is minimal and we can state that as $|p_G| = |p_H|$, $\text{ged}\,(G, H) = \text{ged}\,(H, G)$

2.3.4 $\forall_{F,G,H}\,\text{ged}\,(F, G) + \text{ged}\,(G, H) \geq \text{ged}\,(F, H)$

Suppose the opposite is true, that is $\exists_{F,G,H}\,\text{ged}\,(F, G)+\text{ged}\,(G, H) < \text{ged}\,(F, H)$, and suppose the minimal edit paths for those transformations are p_{FG},p_{GH},p_{FH} ($|p_{FH}| > |p_{FG}| + |p_{GH}|$). However, we take note that by applying first the operations of p_{FG}, we achieve graph F' isomorphic to G. If we apply operations of p_{GH} to F', we will obtain F'' which is isomorphic to H. Then, we can concatenate those two paths to obtain path $p_{FG,GH}$ which directly constructs F'' isomorphic to H from F. The length of $p_{FG,GH}$ is the sum of the lengths of p_{FG} and p_{GH}, which is stated to be less than the length of p_{FH}. However, it contradicts our initial assumption that p_{FH} is minimal with respect to length, meaning there exist no such p_{FG} and p_{GH}. Hence, the opposite must be true and $\forall_{F,G,H}\,\text{ged}\,(F, G) + \text{ged}\,(G, H) \geq \text{ged}\,(F, H)$.

2.4 Graph Isomorphism

An isomorphism between graphs G and H is a bijective mapping $f : V(G) \to V(H)$ such that $\forall_{u,v \in V(G)} \operatorname{mult}_{\mathrm{G}}\left((u,v)\right) = \operatorname{mult}_{\mathrm{H}}\left(\left(\phi\left(u\right), \phi\left(v\right)\right)\right)$ This condition ensures that the mapping is structure-preserving, meaning it maintains the exact adjacency and edge multiplicity between all pairs of vertices. We say that f is a subgraph isomorphism for H and G if there exists a subgraph S of H such that f is an isomorphism between G and S.

2.5 Subgraph Relation

We say G is a subgraph of H if there exists an injective mapping $\phi : V(G) \to V(H)$ such that:

$$\bigwedge_{u,v \in V(G)} \operatorname{mult}_{\mathrm{G}}\left((u,v)\right) \leq \operatorname{mult}_{\mathrm{H}}\left(\left(\phi(u), \phi(v)\right)\right)$$

If such a mapping exists, $G \subseteq H$. If not, we can quantify the deficit of a mapping ϕ:

$$\operatorname{deficit}(\phi) = \sum_{u,v \in V(G)} \max(0, \operatorname{mult}_{\mathrm{G}}\left((u,v)\right) - \operatorname{mult}_{\mathrm{H}}\left(\left(\phi(u), \phi(v)\right)\right))$$

The minimal deficit gives the smallest number of edge insertions required for G to fit inside H.

2.6 Minimal Extension

The minimal extension H' of H is defined as the smallest multigraph such that:

$$H' = (U, F \cup F_{\text{added}}), \quad \text{and} \quad G \subseteq H'$$

with

$$|F_{\text{added}}| = \min_{\phi} \operatorname{deficit}(\phi)$$

That is, H' is obtained by adding the minimal number of missing edges required under the optimal vertex mapping.

3 Checking if there exist at least N distinct subgraph isomorphisms of H and G

3.1 Algorithmic Methodology for Subgraph Isomorphism

The method implements a search algorithm to solve the subgraph isomorphism problem for directed multigraphs. The objective is to enumerate N distinct subgraphs of H such that for each of them there exists an isomorphism with

G. This is an extension of a subgraph isomorphism problem, which answers the question whether there exists at least one such subgraph. As the subgraph isomorphism problem is known to be NP-complete, the same applies to this problem.

The algorithm is split into two primary phases: a pruning phase to reduce the combinatorial search space, followed by a recursive backtracking search.

1. Search Space Reduction via Refinement This phase establishes a set of necessary conditions for any potential mapping, represented by a boolean candidate matrix $M \in \{0,1\}^{|V(G)|\times|V(H)|}$. This phase includes:

1. **Initialization of the Candidate Matrix:** The matrix M is initialized based on a fundamental graph-theoretic constraint: a vertex $i \in V(G)$ can only be mapped to a vertex $j \in V(H)$ if the degrees of i do not exceed the degrees of j.

$$M_{ij} = 1 \iff \text{indeg}_{\text{G}}(i) \leq \text{indeg}_{\text{H}}(j) \wedge \text{outdeg}_{\text{G}}(i) \leq \text{outdeg}_{\text{H}}(j)$$

   If $M_{ij} = 0$, the pair (i, j) is excluded from any further consideration.

2. **Iterative Neighbourhood Refinement:** The algorithm applies an iterative refinement procedure, analogous to achieving local consistency. This procedure prunes the matrix M by enforcing a stronger neighbourhood constraint. A candidate mapping $M_{ij} = 1$ is invalidated (set to 0) if a corresponding match for one of i's neighbours cannot be found among the neighbours of j.

   Formally, M_{ij} remains 1 if and only if both of the following conditions hold:

   - $\forall u \in V(G)$ s.t. $(i,u) \in E(G), \exists v \in V(H)$ s.t. $(j,v) \in E(H) \wedge M_{uv} = 1 \wedge \text{mult}_{\text{G}}\big((i,u)\big) \leq \text{mult}_{\text{H}}\big(\big(\big)j,v\big)$
   - $\forall u \in V(G)$ s.t. $(u,i) \in E(G), \exists v \in V(H)$ s.t. $(v,j) \in E(H) \wedge M_{uv} = 1 \wedge \text{mult}_{\text{G}}\big((u,i)\big) \leq \text{mult}_{\text{H}}\big((v,j)\big)$

   This refinement is applied iteratively until M reaches a fixpoint (i.e., no further entries can be pruned). If at any point there exists $i \in V(G)$ such that $\sum_{j\in V(H)} M_{ij} = 0$, the algorithm terminates, as no isomorphism is possible.

2. Search via Backtracking If the pruning phase does not trivially reject the instance, the algorithm proceeds with a depth-first search to find explicit mappings in the following order:

1. **Heuristic Vertex Ordering:** To optimize the search, a vertex ordering heuristic is employed. The vertices of G are sorted to create an ordered sequence $\sigma = (v_1, v_2, \ldots, v_n)$ where $n = |V(G)|$, such that $\deg_{\text{G}}(v_k) \geq \deg_{\text{G}}\big(v_{k+1}\big)$ for $1 \leq k < n$. The search attempts to find mappings $\phi(v_k)$ in

this sequence. This heuristic prioritizes more constrained vertices, which often leads to earlier pruning of invalid search branches.

2. **Recursive Search:** The core of the algorithm is a recursive function `Dfs(`t`)` that attempts to extend a partial injective mapping $\phi_t : \{v_1, \ldots, v_t\} \to V(H)$ to a mapping ϕ_{t+1} for the vertex v_{t+1}.

   The state of the search at step t is defined by the partial map ϕ_t and its image, $U_t = \text{Image}(\phi_t) \subset V(H)$. To find a mapping for v_{t+1}, the algorithm iterates through all candidate vertices $j \in V(H) \setminus U_t$ for which $M_{v_{t+1},j} = 1$.

3. **Mapping Consistency Verification:** For each such candidate j, the algorithm validates the new mapping $\phi_{t+1} = \phi_t \cup \{v_{t+1} \to j\}$. This mapping is consistent if and only if it preserves the adjacency and multiplicity relations between v_{t+1} and all previously mapped vertices $\{v_1, \ldots, v_t\}$. That is, for all $k \in \{1, \ldots, t\}$:

$$\begin{aligned} & \text{mult}_{\text{G}}\left((v_k, v_{t+1})\right) \leq \text{mult}_{\text{H}}\left((\phi_t(v_k), j)\right) \\ \wedge \quad & \text{mult}_{\text{G}}\left((v_{t+1}, v_k)\right) \leq \text{mult}_{\text{H}}\left((j, \phi_t(v_k))\right) \end{aligned}$$

4. **Recursion and Termination:** If the consistency check is satisfied, the algorithm recursively calls `Dfs(`$t+1$`)`. If this call returns `true` (indicating N solutions have been found), the success propagates up.

   If the check fails, or if the recursive call returns `false` (indicating the branch was exhausted), the algorithm backtracks: j is removed from U_t, the mapping for v_{t+1} is unassigned, and the loop proceeds to the next candidate j.

   The base cases for the recursion are:

   - If $t = |V(G)|$, a complete isomorphic mapping ϕ has been found. The solution count is incremented, and the function returns `false` to continue the search for other solutions.
   - If the solution count reaches N, the function returns `true`, terminating the entire search.

   If the initial call `Dfs(0)` completes without the count reaching N, the algorithm has exhaustively proven that fewer than N such isomorphisms exist.

3.2 Pseudocode

```
1: procedure FindNSubgraphs(G, H, N)
2:     if |V(G)| = 0 or |V(G)| > |V(H)| then return true
3:     end if
4:     M ← InitialCandidates(G, H)
5:     if not RefineCandidatesMultiplicity(G, H, M) then return false
6:     end if
7:     order ← SortVerticesByDescDegree(G)
8:     used_H ← ∅
9:     mapping ← ∅
10:    count ← 0
11:    function Dfs(t)
12:        if count ≥ N then
13:            return true
14:        end if
15:        if t = |V(G)| then
16:            count ← count + 1
17:            return false
18:        end if
19:        i ← order[t]
20:        for all j ∈ V(H) where M[i][j] and j ∉ used_H do
21:            if ConsistentPartial(mapping ∪ {i → j}) then
22:                mapping[i] ← j
23:                used_H ← used_H ∪ {j}
24:                if Dfs(t + 1) then return true
25:                end if
26:                used_H ← used_H \ {j}
27:                remove mapping[i]
28:            end if
29:        end for
30:        return false
31:    end function
32:    Dfs(0)
33: end procedure
```

3.3 Computational Complexity

Let $n_G = |V(G)|$ and $n_H = |V(H)|$. The overall complexity is dominated by the recursive backtracking search, as the initial pruning phase is polynomial.

The algorithm explores a state-space search tree, where the goal is to find an injective mapping $\phi : V(G) \to V(H)$.

- **Search Tree Depth:** The depth of the recursion is n_G, as the algorithm must find a mapping for each vertex in G.
- **Branching Factor:** At any depth t in the tree (attempting to map vertex $v_t \in V(G)$), the algorithm may branch up to n_H times (once for each potential $j \in V(H)$).
- **Work per Node:** At each node at depth t, the algorithm performs a consistency check against the t previously mapped vertices. This check takes $O(t)$ time, which is bounded by $O(n_G)$.

In the worst case, the search must explore all possible partial mappings. The number of nodes in the search tree can be bounded by $O(n_H^{n_G})$. Multiplying by the work per node, $O(n_G)$, gives a loose worst-case time complexity of $O(n_G \cdot n_H^{n_G})$.

A tighter (but still exponential) bound considers the decreasing branching factor: the number of leaves in the search tree is the number of injective mappings, $P(n_H, n_G) = \frac{n_H!}{(n_H - n_G)!}$. The total complexity is related to $O(n_G \cdot P(n_H, n_G))$.

The complexity is therefore exponential in n_G, the size of the pattern graph G. The vertex ordering heuristic is designed to prune large branches of this search tree as early as possible, which improves average-case performance significantly but does not change the worst-case complexity.

4 Methodology for N Minimal Mappings

The `MinimalExtensionN` procedure is designed to determine a minimal composite extension of H that simultaneously accommodates N distinct embeddings of G. Its main goal is to find the smallest edge augmentation to H required such that at least N injective mappings $\phi_i : V(G) \to V(H)$ are simultaneously realizable.
The algorithm operates in two principal stages. First, it enumerates all injective mappings from $V(G)$ to $V(H)$, recording the corresponding edge deficit patterns. In the second stage, it performs a combinatorial search over these recorded deficits to determine the minimal combined augmentation that satisfies N mappings simultaneously.

4.1 Enumeration of All Injective Mappings

This stage performs an exhaustive depth-first search (DFS) to generate all injective vertex mappings $\phi : V(G) \to V(H)$, such as in Section 3.

Deficit matrix computation. For a completed mapping ϕ, the algorithm computes a deficit matrix E_ϕ of size $|V(H)| \times |V(H)|$, defined as:

$$E_\phi\left(i, j\right) = \max\left(0,\, \mathrm{mult_G}\left((a, b)\right) - \mathrm{mult_H}\left(\left(\phi\left(a\right), \phi\left(b\right)\right)\right)\right)$$

for all ordered pairs $(a, b) \in V(G) \times V(G)$. Each matrix E_ϕ represents the number of edges that must be added between $\phi(a)$ and $\phi(b)$ in H to realize G under ϕ. All such matrices are stored in the global set $\mathcal{E} = \{E_{\phi_1}, E_{\phi_2}, \dots\}$.

4.2 Combination Search for N-Minimal Extension

Having enumerated all mapping-induced deficit matrices, the algorithm proceeds to identify a subset of N of them whose combined augmentation cost is minimal.

Matrix combination. The function `adj_matrix_add` merges two deficit matrices A and B elementwise as:

$$(A \oplus B)[i, j] = \max\left(A\,[i, j]\,, B\,[i, j]\right)$$

This reflects that the union of edge requirements cannot reduce any individual multiplicity constraint.

Cost evaluation. For any combined deficit matrix M, the associated cost function is:

$$\text{cost}(M) = \sum_{i=1}^{|V(H)|} \sum_{j=1}^{|V(H)|} M[i, j]$$

representing the total number of edge additions required in H.

Recursive edge selection. The second DFS, `edge_dfs(level)`, recursively selects N distinct elements from $\mathcal{E}$, combining their deficits cumulatively into a running sum matrix. Whenever N mappings have been combined, the resulting cost is compared with the current best; if it is smaller, the algorithm updates both the best cost and corresponding edge set.

4.3 Pseudocode

```
1: function MinimalExtensionN(G, H, N)
2:     order ← SortVerticesByDescDegree(G)
3:     used_H ← ∅
4:     mapping ← ∅
5:     all_edgesets ← ∅
6:
7:     function FullEdgeset(mapping)
8:         local_edgeset ← zero matrix of size |V(H)| × |V(H)|
9:         for all ordered pairs (a, b) ∈ V(G) do
10:            j_a ← mapping[a];   j_b ← mapping[b]
11:            total ← max(0, mult_G((a, b)) − mult_H((j_a, j_b)))
12:            if total > 0 then
13:                local_edgeset[j_a][j_b] ← total
14:            end if
15:        end for
16:        all_edgesets.add(local_edgeset)
17:        return
18:    end function
19:
20:    function dfs(t)
21:        if t = |V(G)| then
22:            full_edgeset(mapping)
23:            return
24:        end if
25:        i ← order[t]
26:        for all j ∈ V(H) where j ∉ used_H do
27:            mapping[i] ← j
28:            used_H ← used_H ∪ {j}
29:            Dfs(t + 1)
30:            used_H ← used_H \ {j}
31:            remove mapping[i]
32:        end for
33:    end function
34:
35:    dfs(0)
36:    best_cost ← 0
37:    best_edgeset ← zero matrix of size |V(H)| × |V(H)|
38:    used_edgesets ← ∅
39:    used_inSum ← zero array of size [N]
40:    sum ← zero matrix of size |V(H)| × |V(H)|
41:
42:    function AdjMatrixAdd(mat1, mat2)
43:        for all i ∈ V(H) do
44:            for all j ∈ V(H) do
```

```
45:                 if mat1[i][j] = 0 and mat2[i][j] ≠ 0 then
46:                     mat1[i][j] ← mat2[i][j]
47:                 end if                ▷ If this triggers, something is inconsistent.
48:                 if mat1[i][j] < mat2[i][j] then
49:                     mat1[i][j] ← mat2[i][j]
50:                 end if
51:             end for
52:         end for
53:         return mat1
54:     end function
55:
56:     function EdgesetCost(mat)
57:         cost ← 0
58:         for all i ∈ V(H) do
59:             for all j ∈ V(H) do
60:                 cost ← cost + mat[i][j]
61:             end for
62:         end for
63:         return cost
64:     end function
65:     function EdgeDfs(level)
66:         cost ← EdgesetCost(sum)
67:         if level = N then
68:             if cost < best_cost then
69:                 best_cost ← cost
70:                 best_edgeset ← copy(sum)
71:             end if
72:         end if
73:         if cost ≥ best_cost then
74:             return
75:         end if
76:         for all i ∈ all_edgesets where all_edgesets[i] ∉ used_edgesets and i ∉
    used_inSum do
77:             used_inSum[level] ← i
78:             sum ← AdjMatrixAdd(sum, all_edgesets[i])
79:             EdgeDfs(level + 1)
80:         end for
81:         return
82:     end function
83:     EdgeDfs(0)
84:     return (best_cost, best_edgeset)
85: end function
```

4.4 Complexity Analysis

Let $n_G = |V(G)|$ and $n_H = |V(H)|$. The complexity arises from two nested combinatorial processes:

- **Mapping Enumeration:** The number of injective mappings from $V(G)$ to $V(H)$ is
$$P(n_H, n_G) = \frac{n_H!}{(n_H - n_G)!}$$
Each mapping requires $O(n_H^2)$ time to compute its deficit matrix, leading to an overall complexity of
$$O\left(P\left(n_H, n_G\right) \cdot n_H^2\right)$$
for the first stage.

- **Combination Search:** The second stage examines combinations of N deficit matrices out of $|\mathcal{E}| = P(n_H, n_G)$ possibilities. In the worst case, this is
$$U\left(\binom{P(n_H, n_G)}{N} \cdot n_H^2\right)$$
since each evaluation requires merging matrices and computing a sum cost. This is considered upper bound, since we can stop calculation for some inherently wrong solutions.

Overall complexity. By basic complexity of **Mapping Enumeration** which is factorial by $P(n_H, n_G)$, whole algorithm is factorial by association.

$$\Omega\left(\frac{n_H!}{(n_H - n_G)!}\right) = O(n_H!)$$

5 Approximation Algorithms

The exact algorithms, due to their exponential complexity, are unfit for large graphs. While we could use a more modern alternative to Ullmann's algorithm, such as VF2 algorithm (Cordella et al. 2004), it does not change the fact that the exact solution will still be of at least exponential order. Thus, we propose two approximation algorithms which offer better time complexities, allowing for reduction of complexity at the cost of reduced accuracy. The basis of both algorithms is to limit the number of iterations by checking only a limited number of possible mappings. In the following section, we define:

1. N as the desired number of subgraph isomorphisms between H and G,
2. K as the limit on the number of checked mappings,

3. SelectMappings(K, G, H) as a function which generates the specified number of distinct mappings, referred to as the selection function.

4. ConstructEdgeSet(G, H, ϕ) as a function which given graphs G, H and a mapping between them, constructs a minimal multiset of edges E_+ such that for $H' = \left(V(H), E(H) \cup E_+\right)$, ϕ encodes a subgraph isomorphism between H' and G. We shall refer to this function as the mapping extension function.

For now, we do not assume any specific selection function. Instead, we shall first define the specific approximation algorithms and the proposed values of K for each, as the way of selection of the edges does not affect the behaviour of each algorithm. Afterwards, we shall explore what properties are desirable for such selection function and propose one of the possible implementations. Whenever we consider the computational complexity of an approximation algorithm, we shall use $O(f(K, G, H))$ as the upper bound for the selection function, as the exact order of the function depends on the exact implementation.

5.1 Mapping Extension Function

5.1.1 Algorithm

To construct the minimum extension of H given mapping ϕ, we can use the algorithm 5.1.1, which iterates over each edge of G and stores the edges (with their multiplicities) which need to be added to the set of edges of H'.

---

function ConstructEdgeSet(G,H,ϕ)
  edgeset $\leftarrow$ empty multiset
  for all $(u, v) \in E(G)$ **do**
    $\mathrm{m} \leftarrow \max\left(\mathrm{mult_G}\left((u, v)\right) - \mathrm{mult_H}\left(\left(\phi(u), \phi(v)\right)\right), 0\right)$
    increase multiplicity of $\left(\phi(u), \phi(v)\right)$ **in** edgeset **by** m
  end for
end function

5.1.2 Computational Complexity

As we iterate over the edges of G, the theoretical computational complexity would be $O\left(\left|E(G)\right|\right)$. We note however, that the common implementation will use adjacency matrix as the implementation of multiset of edges, in which case the complexity is better represented as $O\left(\left|V(G)\right|^2\right)$. As the worse of the two, we will use the bound $O\left(\left|V(G)\right|^2\right)$.

5.2 Isomorphism Verification Function

5.2.1 Algorithm

This function is vastly similar to the mapping extension function. It also enumerates the edges of G and compares their multiplicities with the multiplicities of corresponding edges of H. However, instead of storing the edges, it returns false the moment it encounters a deficit.

---

function DETECTISOMORPHISM(G,H,ϕ)
  for all $(u, v) \in E(G)$ **do**
    if $\text{mult}_{\text{G}}\left((u, v)\right) > \text{mult}_{\text{H}}\left(\left(\phi\left(u\right), \phi\left(v\right)\right)\right)$ **then**
      return false
    end if
  end for
  return true
end function

5.2.2 Computational Complexity

The computational complexity for this algorithm is exactly the same as in case of the mapping extension function, hence we assume it is $O\left(\left|V(G)\right|^2\right)$. However, it will in general perform fewer operations than the mapping extension function thanks to the early return. It also does not store the edges which require adding, so the space complexity is also reduced to $O(1)$ (as opposed to $O\left(\left|V(G)\right|^2\right)$ for the mapping extension function).

5.3 Approximation algorithm for existence of at least N distinct subgraph isomorphisms between H and G

5.3.1 Algorithm

Given limit K, this algorithm shall use the selection function to produce K distinct mappings. Next, for each mapping, it shall verify if it encodes an isomorphism, counting the total number. Lastly, it will return whether the number of found isomorphisms exceeds N.

5.3.2 Computational Complexity

The computational complexity of the approximation algorithm is

$$O\left(f(K, G, H)\right) + K \cdot O\left(\left|V(G)\right|^2\right)$$

or, equivalently,

$$O\left(\max\left(f(K, G, H), \left|V(G)\right|^2 K\right)\right)$$

```
function CheckApprox(K, N, G, H)
    total ← 0
    mappings ← SelectMappings(K,G,H)
    for all φ ∈ mappings do
        if CheckForIsomorphism(G,H,φ) then
            increase total by 1
            if total > N then
                return true
            end if
        end if
    end for
    return false
end function
```

. The first term, $O\left(f(K,G,H)\right)$ is the computational cost of finding K mappings, and it depends on the choice of selection function. The second term, $K \cdot O\left(\left|V(G)\right|^2\right) = O\left(\left|V(G)\right|^2 K\right)$ is the cost of performing the isomorphism verification function for each of the K generated mappings, where each call of said function is $O\left(\left|V(G)\right|^2\right)$.

5.4 Interpretation of results

The algorithm provides an answer to the question: ,,Are there at least N different subgraph isomorphisms between H and G" in form of true (,,Yes") or false (,,No"). However, as an approximation algorithm, it is impossible for it to be 100% accurate (as opposed to the exact algorithms). Thus, only the answer ,,Yes" is fully reliable (as it implies that the algorithm did in fact find N such subgraphs). The answer ,,No" on the other hand implies solely that the algorithm has not found a proper solution, meaning it could yield false negatives. Thus, the result false would be better interpreted as ,,I do not know", in which case, if we require correct answer, we should repeat the approximation with increased limit K, or employ the exact algorithms. Thus the algorithm is primarily useful for the initial verification of the problem, potentially saving time for a subset of possible pairs graphs.

5.5 Choice of K

Obviously, K should be at least N, as otherwise it would be impossible to find sufficiently many isomorphisms. Secondly, the larger K is, the more reliable the answer of the approximation algorithm is.

5.6 Approximation algorithm for finding the extension of H

5.6.1 Algorithm

```
1: function ExtendApprox(K, G, H, N)
2:     mappings ← SelectMappings(K,G,H)
3:     fullEdgeSet ← empty multiset of edges
4:     for all φ ∈ mappings do
5:         currentEdgeSet ← ConstructEdgeSet(G,H,φ)
6:         fullEdgeSet ← MergeEdgeSets(fullEdgeSet,currentEdgeSet)
7:     end for
8:     return fullEdgeSet
9: end function
10:
11: function MergeEdgeSets(L,R)
12:     for all (u, v) ∈ R do
13:         set multiplicity of (u, v) in L set Max(mult_L ((u, v)),mult_R ((u, v)))
14:     end for
15:     return M
16: end function
```

5.6.2 Computational Complexity

First, the computational complexity of MergeEdgeSets(L, R) is $O(|R|)$, note that the argument R in our case will be the constructed multiset of edges. The wors case scenario is when the generated set of edges increases the multiplicity of every possible edge, of which there is $|V(G)|$, hence in the case of the presented algorithm, the function MergeEdgeSets is $O(|V(G)|)$.

Then, the total complexity of the algorithm is

$$O\left(f(K,G,H)\right) + K \cdot \left(O\left(\left|V(G)\right|^2\right) + O\left(\left|V(G)\right|\right)\right)$$

which can be simplified to

$$O\left(\max\left(f(K,G,H)\right), O\left(\left|V(G)\right|^2 K\right)\right)$$

The first term, $O\left(f(K,G,H)\right)$ is once again the computational cost of finding K mappings, dependent on the choice of selection function. The second term, $K \cdot \left(O\left(\left|V(G)\right|^2\right) + O\left(\left|V(G)\right|\right)\right)$ is the cost of performing for each of the K generated mappings:

1. mapping expansion function, which is $O(|V(G)|^2)$

2. merging of edge sets, which is $O(|V(G)|)$ as mentioned above

The quadratic part overtakes the linear, hence we omit it in the final statement on the complexity.

5.7 Selection Algorithm

Lastly, it is necessary to decide on the selection function. For it to be viable for our approximation algorithms, we need it to conform to several goals/requirements.

First of all, as the primary goal of the approximation algorithm is to be less computationally expensive than the exact algorithm, it is the preliminary requirement that the selection function is less than exponential.

The second goal for the function should be to select the mappings which are more likely to constitute an isomorphism, which can make them yield results closer to the exact solution. For brevity, we shall refer to this likelihood as the quality of a mapping.

Lastly, the function should be required to be deterministic. That is, the returned set of mappings should always be the same given the same set of inputs. Otherwise, running the same algorithm several times would possibly yield different results, makes it difficult to reason about.

There are several possibilities for the choice of selection function. One would be to completely ignore the second goal (estimation of quality of mappings), to focus on the first one - in which case one could utilise a specific pattern of mappings (e.g. first mapping is defined by the diagonal of a matrix, the second one is found by swapping the mappings of the 1st and 3rd mapping of vertices of G, etc.). This would possibly let the approximation algorithms run with quadratic time complexity with respect to $|V(G)|$. However, the accuracy of the approximations could be vastly different depending on the provided graphs.

A different algorithm would be to treat the mappings as the k-best assignments problem, a generalisation of the assignment problem. There are several possible implementations of an algorithm to solve this problem - the one we propose is a Murty's algorithm based on the Hungarian method.

The Hungarian method is an algorithm which, given a cost matrix, calculates the optimal assignment by minimising the cost. It does so by performing row and column reductions to find potential mappings and finding the minimal coverings using said candidates. The Hungarian algorithm is known to run in strictly polynomial time, with the proper basic implementation being $O(n^4)$ or, for the optimised implementation $O(n^3)$. In our case, n is the size of the larger set of vertices, which is $V(H)$, hence we claim that for our algorithm, the computational complexity of Hungarian method is $O\left(\left|V(H)\right|^2\right)$.

The Murty's algorithm (Murty 1968) is an extension of a Hungarian method (Kuhn 1955) to find K best assignments. It begins with the traditional Hungarian method to find the best solution. Then, it repeatedly modifies the cost matrix by disallowing some of the possible vertex mappings to find a new optimal solution (the choice of which mappings to disallow is such that the increase in total cost is minimal). The found solutions are placed on

a priority queue to efficiently sort them by their relative total costs. The computational complexity of the Murty's algorithm for K best assignments is $K \cdot O(|V(H)|) \cdot O(\text{Hungarian}(G, H))$, which is $O\left(K|V(H)|^5\right)$ or $O\left(K|V(H)|^4\right)$ depending on which implementation of the Hungarian algorithm is used.

In our case, the cost is defined as the opposite of quality, where quality is measured based on the simple observation that for each pair of vertices $u \in G$ and $v \in H$, the greater $\deg_{\text{H}}(v) - \deg_{\text{G}}(u)$ is (that is, how many incident edges v has in H compared to u in G), the more likely it is that we won't need to include additional edges in the mapping of those two vertices.

```
1: function VertexMappingCost(G,H)
2:     cost ← |V(G)| × |V(H)| matrix, default 0
3:     for all u ∈ V(G), v ∈ V(H) do
4:         cost[u,v] ← deg_G(u) − deg_H(v)
5:     end forreturn cost
6: end function
```

References

Cordella, Luigi P., Pasquale Foggia, Carlo Sansone, and Mario Vento. 2004. "A (sub)graph isomorphism algorithm for matching large graphs." *IEEE Transactions on Pattern Analysis and Machine Intelligence* 26 (10): 1367–1372.

Garey, Michael R., and David S. Johnson. 1979. *Computers and Intractability: A Guide to the Theory of NP-Completeness.* W. H. Freeman.

Kuhn, H. W. 1955. "The Hungarian method for the assignment problem." *Naval Research Logistics Quarterly* 2 (1-2): 83–97. https://doi.org/10.1002/nav.3800020109.

Murty, K. G. 1968. "An algorithm for ranking all the assignments in order of increasing cost." *Operations Research* 16 (3): 682–687. https://doi.org/10.1287/opre.16.3.682.

Sanfeliu, A., and K.-S. Fu. 1983. "A distance measure between attributed relational graphs for pattern recognition." *IEEE Transactions on Systems, Man, and Cybernetics* SMC-13 (3): 353–362. https://doi.org/10.1109/TSMC.1983.6313167.

Ullmann, J. R. 1976. "An algorithm for subgraph isomorphism." *Journal of the ACM (JACM)* 23 (1): 31–42. https://doi.org/10.1145/321921.321925.

Valiant, Leslie G. 1979. "The Complexity of Computing the Permanent." *Theoretical Computer Science* 8 (2): 189–201.